

Locality-Sensitive Hashing Scheme based on Longest Circular Co-Substring

Yifan Lei

National University of Singapore
lei yifan@u.nus.edu

Mohan Kankanhalli

National University of Singapore
mohan@comp.nus.edu.sg

Qiang Huang*

National University of Singapore
huangq@comp.nus.edu.sg

Anthony K. H. Tung

National University of Singapore
atung@comp.nus.edu.sg

ABSTRACT

Locality-Sensitive Hashing (LSH) is one of the most popular methods for c -Approximate Nearest Neighbor Search (c -ANNS) in high-dimensional spaces. In this paper, we propose a novel LSH scheme based on the Longest Circular Co-Substring (LCCS) search framework (LCCS-LSH) with a theoretical guarantee. We introduce a novel concept of LCCS and a new data structure named Circular Shift Array (CSA) for k -LCCS search. The insight of LCCS search framework is that close data objects will have a longer LCCS than the far-apart ones with high probability. LCCS-LSH is *LSH-family-independent*, and it supports c -ANNS with different kinds of distance metrics. We also introduce a multi-probe version of LCCS-LSH and conduct extensive experiments over five real-life datasets. The experimental results demonstrate that LCCS-LSH outperforms state-of-the-art LSH schemes.

ACM Reference Format:

Yifan Lei, Qiang Huang, Mohan Kankanhalli, and Anthony K. H. Tung. 2020. Locality-Sensitive Hashing Scheme based on Longest Circular Co-Substring. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD'20)*, June 14–19, 2020, Portland, OR, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3318464.3389778>

1 INTRODUCTION

Nearest Neighbor Search (NNS) is a fundamental problem, and it has wide applications in various fields, such as data

mining, multimedia databases, machine learning, and artificial intelligence. Given a distance metric, a database $\mathcal{D}$ of n data objects and a query q with feature representation in d -dimensional space $\mathbb{R}^d$, the aim of NNS is to find the object $o^* \in \mathcal{D}$ which is closest to q , where o^* is called the Nearest Neighbor (NN) of q . The exact NNS in low-dimensional spaces has been well solved by tree-based methods [6, 18, 26]. For high-dimensional NNS, due to the difficulty of finding exact solutions [20, 39], the approximate version of NNS, named c -Approximate NNS (c -ANNS), has been widely studied in recent two decades [7, 13, 14, 23–25, 27, 32, 35, 38, 41].

Prior Work. Locality-Sensitive Hashing (LSH) [19, 23] and its variants [2, 4, 9–11, 15, 16, 22, 28, 34] are one of the most popular methods for high-dimensional c -ANNS. An LSH scheme consists of two components: the LSH function family (or simply LSH family) and the search framework. The idea of LSH families is to construct a family of hash functions such that the *positive probability* p_1 of the close objects to be hashed into the same bucket with a query q is higher than the *negative probability* p_2 of the far-apart ones. Furthermore, the search framework aims to increase the gap between p_1 and p_2 , so that the close objects can be identified efficiently. The most popular search frameworks are the static concatenating search framework [11, 30, 31, 36] and the dynamic collision counting framework [15, 21, 22, 40].

Static Concatenating Search Framework. The static concatenating search framework was first introduced by Indyk et al. [23] for Hamming distance, and later was extended to l_p distance ($0 < p \leq 2$) by Datar et al. [11], which led to E2LSH [1] for Euclidean distance ($p = 2$). E2LSH adopts this framework as follows. In the indexing phase, E2LSH concatenates K i.i.d. LSH functions $h_1, h_2, \dots, h_K$ to form a compound hash function G , i.e., $G(o) = (h_1(o), h_2(o), \dots, h_K(o))$ for all $o \in \mathcal{D}$. If two objects o and q have the same hash value, i.e., $G(o) = G(q)$, we say o and q *collide* in the same bucket under G . E2LSH samples uniformly at random L such hash functions $G_1(\cdot), G_2(\cdot), \dots, G_L(\cdot)$ and builds L hash tables. In the query phase, E2LSH computes L hash values

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD'20, June 14–19, 2020, Portland, OR, USA

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-6735-6/20/06...\$15.00

<https://doi.org/10.1145/3318464.3389778>

	h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8		h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8		h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8
o_1	1	2	4	5	6	6	7	8	o_1	1	2	4	5	6	6	7	8	o_1	1	2	4	5	6	6	7	8
o_2	5	2	2	4	3	6	7	8	o_2	5	2	2	4	3	6	7	8	o_2	5	2	2	4	3	6	7	8
o_3	3	1	3	5	5	6	4	9	o_3	3	1	3	5	5	6	4	9	o_3	3	1	3	5	5	6	4	9
q	1	2	3	4	5	6	7	8	q	1	2	3	4	5	6	7	8	q	1	2	3	4	5	6	7	8

(a) E2LSH (b) C2LSH (c) LCCS-LSH

Figure 1: An example of the search frameworks of E2LSH, C2LSH, and LCCS-LSH

$G_1(q), G_2(q), \dots, G_L(q)$ and lookups the corresponding L buckets to find the candidates of q . The variants of E2LSH, such as LSH-Forest [5], Multi-Probe LSH [31], LSB-Forest [36], and SK-LSH [30], follow this search framework.

Notably, E2LSH conducts the c -ANNS with sublinear time $O(dn^\rho \log_{1/p_2}(n))$, where $\rho = \ln(1/p_1)/\ln(1/p_2)$ [11]. The reason is that the static concatenating search framework can effectively avoid the false positives in the sense that the far-apart objects hardly collide with q . Due to the use of K concatenated LSH functions, such negative probability decreases significantly from p_2 to p_2^K . However, the positive probability also decreases significantly from p_1 to p_1^K , and hence the true positives are not easy to be identified neither. For example, as shown in Figure 1(a), suppose o_1 is the NN of q , o_2 is also close to q , while o_3 is far-apart from q . We consider $K = 4$ and $L = 2$. Due to the use of this framework, o_3 does not collide with q , but the close objects o_1 and o_2 also fail to collide with q . To achieve a certain recall, the number of hash tables (i.e., L) of E2LSH is often set to be more than one hundred, and sometimes up to several hundred [15], leading to a large amount of indexing overhead.

Dynamic Collision Counting Framework. To reduce the large indexing overhead, Gan et al. [15] introduced a dynamic collision counting framework and the C2LSH scheme accordingly. In the indexing phase, C2LSH uses m independent LSH functions $h_1, h_2, \dots, h_m$ to build m hash tables individually. Two objects o and q collide in the same bucket under h if $h(o) = h(q)$. The idea of C2LSH is that, if o is close to q in the original space $\mathbb{R}^d$, then o and q will collide frequently among the m hash tables. Thus, in the query phase, C2LSH maintains the collision number $\#Col(o)$ for each o which collides with q , and o is considered as an NN candidate of q if $\#Col(o) \geq l$, where l is the collision threshold. C2LSH returns the final answers from a set of such candidates. In fact, this framework can be considered as a *dynamic l -concatenating search framework*, because it checks a candidate o until $\#Col(o) \geq l$. Compared to the static concatenating search framework which uses KL LSH functions to generate L combinations only, this framework can generate $\binom{m}{l}$ combinations for each o . Thus, for the same recall, C2LSH requires much less number of

LSH functions than E2LSH, and thus takes much less indexing overhead. Various extensions, such as QALSH [21, 22] and LazyLSH [40], are proposed based on this framework.

However, the query time complexity of C2LSH in the worst case is $O(n \log n)$ [15], which limits its scalability for large n . Notice that C2LSH builds hash tables for every single LSH function. Even though p_2 is small for the far-apart objects, there are expected $(1 - (1 - p_2)^m)n \approx p_2 mn$ objects with at least one collision, which cannot be neglected especially for large n . For example, as shown in Figure 1(b), suppose $m = 8$ and $l = 4$, C2LSH can identify the close objects o_1 and o_2 since $\#Col(o_1) = \#Col(o_2) > l$, but it also conducts 3 times collision counting for the far-apart object o_3 .

Our Method. To achieve a better trade-off between space and query time, we introduce a novel LSH scheme based on the *Longest Circular Co-Substring* (LCCS) search framework (LCCS-LSH). We first introduce a novel concept of LCCS and a new data structure named Circular Shift Array (CSA) for k -LCCS search. Then, in the indexing phase, we exploit a collection of m independent LSH functions $h_1, h_2, \dots, h_m$ to convert data objects into hash strings of length m , i.e., $H(o) = [h_1(o), h_2(o), \dots, h_m(o)]$. The insight is that, if o is close to q in $\mathbb{R}^d$, then $H(o)$ will have a longer LCCS with $H(q)$ than the hash strings for the far-apart ones with high probability. Let $|LCCS(H(o), H(q))|$ be the length of LCCS between $H(o)$ and $H(q)$. In the query phase, we find data objects with the largest $|LCCS(H(o), H(q))|$ as candidates of q and get the final answers from a set of such candidates. For example, as shown in Figure 1(c), suppose $m = 8$ and we combine the 8 hash values for each object as a circular hash string. $|LCCS(H(o_1), H(q))| = 5$, which is larger than $|LCCS(H(o_2), H(q))|$ and $|LCCS(H(o_3), H(q))|$, which are 3 and 2, respectively. Thus, the NN o_1 can be determined efficiently. Furthermore, since LCCS-LSH works on the hash strings only, which is independent of data types, it is *LSH-family-independent* and can be applied to c -ANNS under different distance metrics that admit LSH families.

Contributions. In this paper, we introduce a novel LSH scheme LCCS-LSH for high-dimensional c -ANNS. The LCCS search framework dynamically concatenates consecutive

hash values for data objects, which can identify the close objects in an efficient and effective manner and it requires to tune only a single parameter m . LCCS-LSH enjoys a quality guarantee on query results, and we further analyse its space and time complexities. In addition, we introduce a multi-probe version of LCCS-LSH to reduce the indexing overhead. Experimental results over five real-life datasets demonstrate that LCCS-LSH outperforms state-of-the-art LSH schemes, such as Multi-Probe LSH and FALCONN.

Organization. The roadmap of the paper is as follows. Section 2 discusses the problem settings. The LCCS search framework is introduced in Section 3. LCCS-LSH and its theoretical analysis are presented in Sections 4 and 5, respectively. Section 6 reports experimental results. Section 7 surveys the related work. Finally, we conclude our work in Section 8.

2 PROBLEM SETTINGS

In this paper, we consider data objects and queries represented as vectors in d -dimensional space $\mathbb{R}^d$. Let $\text{Dist}(o, q)$ be a distance metric between any two objects o and q . Suppose $\mathcal{D}$ is a database of n data objects from $\mathbb{R}^d$. Given a query q , we say o^* is the Nearest Neighbor (NN) of q such that $o^* = \arg \min_{o \in \mathcal{D}} \text{Dist}(o, q)$. Then,

Definition 2.1 (c -ANNS): Given an approximation ratio c ($c > 1$), the problem of c -ANNS is to construct a data structure which, for any query $q \in \mathbb{R}^d$, finds a data object $o \in \mathcal{D}$ such that $\text{Dist}(o, q) \leq c \cdot \text{Dist}(o^*, q)$, where $o^* \in \mathcal{D}$ is the NN of q .

Similarly, the problem of c - k -ANNS is to construct a data structure which, for any query $q \in \mathbb{R}^d$, finds k data objects $o_i \in \mathcal{D}$ ($1 \leq i \leq k$) such that $\text{Dist}(o_i, q) \leq c \cdot \text{Dist}(o_i^*, q)$, where $o_i^* \in \mathcal{D}$ is the i^{th} NN of q .

LSH schemes [2, 10, 11, 23] cannot solve the problem of c -ANNS directly. Instead, they solve the problem of (R, c) -Near Neighbor Search ((R, c) -NNS), which is a decision version of c -ANNS. One can reduce the c -ANNS problem to a series of (R, c) -NNS via a binary-search-like method within a log factor overhead, where $R \in \{1, c, c^2, \dots\}$. Formally,

Definition 2.2 ((R, c) -NNS): Given a search radius R ($R > 0$) and an approximation ratio c ($c > 1$), the problem of (R, c) -NNS is to construct a data structure which, for any $q \in \mathbb{R}^d$, returns objects that satisfy the following conditions:

- If there is an object $o \in \mathcal{D}$ such that $\text{Dist}(o, q) \leq R$, then return an arbitrary object o' such that $\text{Dist}(o', q) \leq cR$;
- If $\text{Dist}(o, q) > cR$ for all $o \in \mathcal{D}$, then return nothing;
- Otherwise, the result is undefined.

The LCCS-LSH scheme is orthogonal to the LSH family and can handle various kinds of distance metrics. Thus, $\text{Dist}(\cdot, \cdot)$ can be the widespread distance metrics. In this paper, we focus on two popular distance metrics, i.e., Euclidean

distance and Angular distance, to demonstrate the superior performance of LCCS-LSH. Notice that we do *not* claim that every distance metric can be handled by LCCS-LSH. It supports the distance metrics if and only if there exist LSH families for them.

3 THE LCCS SEARCH FRAMEWORK

3.1 Definition of LCCS

We first introduce the definition of *Circular Co-Substring*. It can be considered as the common circular substring of two strings starting from the same position. Formally,

Definition 3.1: Given two strings $T = [t_1, t_2, \dots, t_m]$ and $Q = [q_1, q_2, \dots, q_m]$ of the same length m , a string X is a Circular Co-Substring of T and Q if and only if X is an empty string, $X = [t_i, t_{i+1}, \dots, t_j] = [q_i, q_{i+1}, \dots, q_j]$, or $X = [t_j, \dots, t_m, t_1, \dots, t_i] = [q_j, \dots, q_m, q_1, \dots, q_i]$, where $1 \leq i < j \leq m$.

Example 3.1: Consider two strings $T = [1, 2, 3, 4, 1, 5]$ and $Q = [1, 1, 2, 3, 4, 5]$ as an example. The substring $[5, 1]$ is a Circular Co-Substring of T and Q . However, although the substring $[1, 2, 3, 4]$ is a common circular substring of T and Q , it is not a Circular Co-Substring, because it does not start from the same position of T and Q . $\triangle$

Let $|T|$ be the length of a string T . The Longest Circular Co-Substring (LCCS) is defined as follows.

Definition 3.2: Given any two strings T and Q of the same length, let $S(T, Q)$ be the set of all Circular Co-Substrings of T and Q . The LCCS of T and Q is defined as $\text{LCCS}(T, Q) = \arg \max_{X \in S(T, Q)} |X|$.

Then, the problem of k -Longest Circular Co-Substring search (k -LCCS search) is defined as follows.

Definition 3.3: Given a collection of strings $\mathcal{T}$ of the same length m , the problem of k -LCCS search is to construct a data structure which, for any query string Q that $|Q| = m$, finds a set of strings $\mathcal{T}^* \subseteq \mathcal{T}$ with cardinality k such that for all $T^* \in \mathcal{T}^*, T' \in \mathcal{T} \setminus \mathcal{T}^*, |\text{LCCS}(T', Q)| \leq |\text{LCCS}(T^*, Q)|$.

3.2 k -LCCS Search

Suppose $\text{LCP}(T, Q)$ is the Longest Common Prefix (LCP) between two strings T and Q . Given a string $T = [t_1, t_2, \dots, t_m]$ and an integer $i \in \{0, 1, \dots, m-1\}$, let $\text{shift}(T, i) = [t_{i+1}, \dots, t_m, t_1, \dots, t_i]$ be the circular string of T after shifting i positions. Since the index of T starts from 1, $\text{shift}(T, i-1)$ corresponds to the circular string of T starting from t_i . For simplicity, for a collection of strings $\mathcal{T}$, we let $\text{shift}(\mathcal{T}, i) = \{\text{shift}(T, i) \mid T \in \mathcal{T}\}$.

To solve the problem of k -LCCS search, we propose a data structure named *Circular Shift Array (CSA)*, which is inspired by Suffix Array [33]. Specifically, the insight of CSA comes from Fact 3.1, which is described as follows.

Algorithm 1: Building CSA

Input: $\mathcal{T}$: a dataset of n strings of length m such that $\mathcal{T} = \{T_1, T_2, \dots, T_n\}$ and $|T_i| = m$.
Output: m sorted indices $\{I_1, I_2, \dots, I_m\}$ and m next links $\{N_1, N_2, \dots, N_m\}$.

```

1 for  $i = 1$  to  $m$  do
2    $I_i = \text{arg sort}(\text{shift}(\mathcal{T}, i - 1));$  ▷  $I_i$  is the sorted index of  $n$  strings in  $\text{shift}(\mathcal{T}, i - 1)$ 
3 for  $i = 1$  to  $m$  do
4   for  $j = 1$  to  $n$  do
5      $\text{pos}[I_{i \% m + 1}[j]] = j;$  ▷  $\text{pos}$  is the position of  $n$  strings in the next sorted index  $I_{i \% m + 1}$ 
6   for  $j = 1$  to  $n$  do
7      $N_i[j] = \text{pos}[I_i[j]];$  ▷  $N_i$  is the position of  $n$  strings in the next sorted  $\text{shift}(\mathcal{T}, i \% m)$ 
8 return  $\{I_1, I_2, \dots, I_m\}$  and  $\{N_1, N_2, \dots, N_m\};$ 

```

Fact 3.1: Given two strings T and Q of the same length m , $\text{LCCS}(T, Q) = \max_{i \in \{0, 1, \dots, m-1\}} \text{LCP}(\text{shift}(T, i), \text{shift}(Q, i))$.

$\text{LCCS}(T, Q)$ can be identified by considering the LCP of all shifted T 's and Q 's. Let $<$ and $\leq$ be the alphabetical order relationships of strings, where $\leq$ allows equal cases. Then,

Fact 3.2: If $T_1 \leq T_2 < T_3$, then $\forall Q$,

$$|\text{LCP}(T_2, Q)| \geq \min(|\text{LCP}(T_1, Q)|, |\text{LCP}(T_3, Q)|).$$

The soundness of Fact 3.2 is obvious. Let $\min(\mathcal{T})$ and $\max(\mathcal{T})$ be the minimum and maximum string in alphabetical order of $\mathcal{T}$, respectively. Then,

Corollary 3.1: For any string Q s.t. $\min(\mathcal{T}) \leq Q < \max(\mathcal{T})$, let $T_l = \arg \max_{T \in \mathcal{T}} T \leq Q$ and $T_u = \arg \min_{T \in \mathcal{T}} Q < T$ be the lower bound and upper bound of Q , respectively. If $T^* = \arg \max_{T \in \mathcal{T}} |\text{LCP}(T, Q)|$, then $T^* = T_l$ or $T^* = T_u$.

Corollary 3.1 indicates that, given a query string Q and a database of n sorted strings $\mathcal{T}$ in alphabetical order, one can use binary search on $\mathcal{T}$ to find T^* in $O(m + \log n)$ time. It yields a simple method with two phases to answer the 1-LCCS query: in the indexing phase, for each $i \in \{1, 2, \dots, m\}$, we sort $\text{shift}(\mathcal{T}, i - 1)$ in alphabetical order and maintain the sorted index I_i ; in the query phase, to find the 1-LCCS of Q , we conduct binary search on each I_i to get T_i^* such that $T_i^* = \arg \max_{T \in \text{shift}(\mathcal{T}, i - 1)} |\text{LCP}(T, \text{shift}(Q, i - 1))|$; the 1-LCCS of Q is the string T^* among $\{T_1^*, T_2^*, \dots, T_m^*\}$ with the largest $|\text{LCP}(T_i^*, \text{shift}(Q, i - 1))|$. This simple method requires m times binary search, and hence the query time complexity is $O(m(m + \log n))$. Next, we introduce a strategy to reduce the complexity to $O(m + \log n)$ under certain assumptions.

Lemma 3.1: Suppose $T_l \leq Q < T_u$. For any $k \geq 1$, if we have $|\text{LCP}(T_l, Q)| \geq k$ and $|\text{LCP}(T_u, Q)| \geq k$, then $\text{shift}(T_l, k) \leq \text{shift}(Q, k) < \text{shift}(T_u, k)$.

Lemma 3.1 is true according to the definition of $<$ and $\leq$. According to Lemma 3.1, once we conduct binary search on $\text{shift}(\mathcal{T}, i - 1)$ for a query string $\text{shift}(Q, i - 1)$ and find $T_{l,i}$

and $T_{u,i}$ as its lower bound and upper bound, respectively, we can immediately know a loose lower bound and a loose upper bound for $\text{shift}(Q, i)$. Let $\text{len}_{l,i} = |\text{LCP}(T_{l,i}, \text{shift}(Q, i - 1))|$ and $\text{len}_{u,i} = |\text{LCP}(T_{u,i}, \text{shift}(Q, i - 1))|$. Then,

Corollary 3.2: If $\text{len}_{l,i} \geq 1$ and $\text{len}_{u,i} \geq 1$, then the lower bound $T_{l,i+1}$ and upper bound $T_{u,i+1}$ of $\text{shift}(Q, i)$ satisfy that $\text{shift}(T_{l,i}, 1) \leq T_{l,i+1} \leq \text{shift}(Q, i) < T_{u,i+1} \leq \text{shift}(T_{u,i}, 1)$.

Based on Lemma 3.1 and Corollary 3.2, the simple method discussed before can be further optimized. To find the 1-LCCS of Q , we conduct only once binary search on the whole $\text{shift}(\mathcal{T}, 0)$ ($i = 1$) for the query $\text{shift}(Q, 0)$ (or simply Q). Then, for the query $\text{shift}(Q, i - 1)$ ($i > 1$), according to Corollary 3.2, we can conduct binary search on $\text{shift}(\mathcal{T}, i - 1)$ between $\text{shift}(T_{l,i-1}, 1)$ and $\text{shift}(T_{u,i-1}, 1)$. After that, we get $T_{l,i}$ and $T_{u,i}$ and can continue to use them to narrow down the binary search range for the next query $\text{shift}(Q, i)$. We repeat this procedure, and the 1-LCCS of Q will be the string with the longest LCP among $\text{LCP}(T_{l,i}, \text{shift}(Q, i - 1))$ and $\text{LCP}(T_{u,i}, \text{shift}(Q, i - 1))$ for all $i \in \{1, 2, \dots, m\}$.

To speed up the query phase, we need to know the positions of $\text{shift}(T_{l,i}, 1)$ and $\text{shift}(T_{u,i}, 1)$ when we get $T_{l,i}$ and $T_{u,i}$. Thus, in the indexing phase, we not only need to maintain the sorted indices $\{I_1, I_2, \dots, I_m\}$, but also require to store the next links $\{N_1, N_2, \dots, N_m\}$, e.g., N_i stores the positions of $\mathcal{T}$ in the next sorted $\text{shift}(\mathcal{T}, i \% m)$. The pseudo-code of building CSA is depicted in Algorithm 1.

To find the k -LCCS of Q , we first compute $T_{l,i}$ and $T_{u,i}$ for each $\text{shift}(Q, i - 1)$. Then, we construct a priority queue PQ and perform a $2m$ -way sorted list merge. The strings with top- k longest lengths in PQ are the k -LCCS results of Q . The pseudo-code of k -LCCS search is shown in Algorithm 2.

Example 3.2: We now use an example to illustrate Algorithm 2. Suppose $k = 1$. We continue to use the same o_1, o_2, o_3 and q from Figure 1 as in Figure 2. We follow Algorithm 1 and build CSA with sorted indices $\{I_1, I_2, \dots, I_8\}$ and next links $\{N_1, N_2, \dots, N_8\}$, e.g., $I_1 = [1, 3, 2]$ and $N_1 = [3, 1, 2]$.

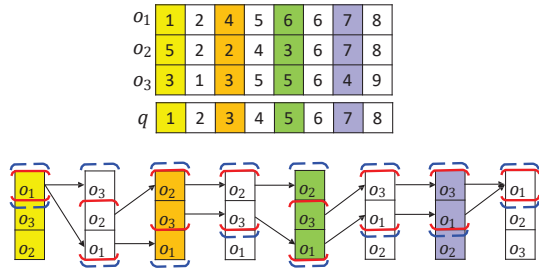
Algorithm 2: k -LCCS Search using CSA

Input: $\mathcal{T} = \{T_1, T_2, \dots, T_n\}, \{I_1, I_2, \dots, I_m\}, \{N_1, N_2, \dots, N_m\}, Q$, and #candidates k ;
Output: C : the results of k -LCCS search.

```

1  $C \leftarrow \emptyset; PQ \leftarrow \emptyset;$                                  $\triangleright C$  is a candidate set and  $PQ$  is a priority queue
2  $(pos_{l,1}, pos_{u,1}, len_{l,1}, len_{u,1}) \leftarrow \text{BinarySearch}(I_1, Q);$    $\triangleright pos_{l,i}$  and  $pos_{u,i}$  are the positions of  $T_{l,i}$  and  $T_{u,i}$  in  $I_i$ 
3  $PQ.\text{push}(len_{l,1}, pos_{l,1}, 1, -1);$                                  $\triangleright -1$  represents the down direction
4  $PQ.\text{push}(len_{u,1}, pos_{u,1}, 1, +1);$                                  $\triangleright +1$  represents the up direction
5 for  $i = 2$  to  $m$  do
6   if  $len_{l,i-1} \geq 1$  and  $len_{u,i-1} \geq 1$  then
7      $(pos_{l,i}, pos_{u,i}, len_{l,i}, len_{u,i}) \leftarrow \text{BinarySearchBetween}(I_i, \text{shift}(Q, i-1), N_{i-1}[I_{i-1}[pos_{l,i-1}]], N_{i-1}[I_{i-1}[pos_{u,i-1}]]);$ 
8   else
9      $(pos_{l,i}, pos_{u,i}, len_{l,i}, len_{u,i}) \leftarrow \text{BinarySearch}(I_i, \text{shift}(Q, i-1));$ 
10   $PQ.\text{push}(len_{l,i}, pos_{l,i}, i, -1);$ 
11   $PQ.\text{push}(len_{u,i}, pos_{u,i}, i, +1);$ 
12 while  $|C| < k$  do
13    $(len, pos, i, dir) \leftarrow PQ.\text{top}(); PQ.\text{pop}();$ 
14    $C \leftarrow C \cup \{I_i[pos]\};$ 
15    $PQ.\text{push}(|\text{LCP}(\text{shift}(T_{l_i}[pos], i-1), \text{shift}(Q, i-1))|, pos + dir, i, dir);$ 
16 return  $C;$ 

```

**Figure 2:** An example of the 1-LCCS search

To find the 1-LCCS of q , we first conduct binary search on the whole I_1 , and get $pos_{l,1}$ and $pos_{u,1}$ as depicted by red brackets (line 2). Since $q < o_1$, $pos_{l,1} = pos_{u,1} = 1$. Then, along with N_i , the search range on the next I_{i+1} can be determined, as shown by blue dash brackets (lines 5–9). For example, consider I_5 , since $\text{shift}(o_3, 4) < \text{shift}(q, 4) < \text{shift}(o_1, 4)$, $pos_{l,5} = 2$ and $pos_{u,5} = 3$. Since $N_5[pos_{l,5}] = 1$ and $N_5[pos_{u,5}] = 2$, the search range on I_6 is narrowed down to $[1, 2]$. We use a priority queue to check the objects with longest LCP among $\{I_1, I_2, \dots, I_8\}$ (lines 3–4, 8–9, and 12–15). o_1 is first verified because o_1 has the largest $|\text{LCP}(\text{shift}(o_1, 5), \text{shift}(q, 5))| = 5$ on I_6 , and o_1 is the 1-LCCS of q . $\triangle$

Theorem 3.1: Let $T = [t_1, t_2, \dots, t_m]$ and $Q = [q_1, q_2, \dots, q_m]$. If the probability that $t_i = q_i$ is independent and equals to p for each i , one can build CSA using Algorithm 1 with $O(nm)$ space and $O(mn \log n)$ time, and answer the k -LCCS queries using Algorithm 2 with $O(\log n + (m + k) \log m)$ time.

The proof of Theorem 3.1 can be referred to [29].

4 THE LCCS-LSH SCHEME

In this section, we present the LCCS-LSH schemes for high-dimensional c -ANNS.

4.1 Single-Probe LCCS-LSH

The single-probe LCCS-LSH scheme (or simply LCCS-LSH) consists of two phases: indexing phase and query phase.

Indexing Phase. Given a database $\mathcal{D}$ of n data objects, LCCS-LSH first generates m independent LSH functions $h_1, h_2, \dots, h_m$ from the LSH family $\mathcal{H}$. Then, it computes the m hash values $h_1(o), h_2(o), \dots, h_m(o)$ for each $o \in \mathcal{D}$ and concatenates all of them to a hash string $H(o) = [h_1(o), h_2(o), \dots, h_m(o)]$ of length m . Let $\mathcal{T} = \{H(o) \mid o \in \mathcal{D}\}$ be a collection of n such hash strings. Finally, LCCS-LSH constructs a data structure CSA for $\mathcal{T}$ using Algorithm 1.

Query Phase. For the c -ANNS of q , LCCS-LSH first computes the hash string $H(q)$. Then, it conducts a λ -LCCS search of $H(q)$ using Algorithm 2, and gets a set C of candidates such that $|C| = \lambda$. Finally, we compute the actual distance between each candidate $o \in C$ and q , and return the nearest one as the c -ANNS answer of q . For the c - k -ANNS of q , LCCS-LSH only needs to conduct $(\lambda + k - 1)$ -LCCS search of $H(q)$ and verifies $(\lambda + k - 1)$ candidates from C accordingly. The nearest k objects among C are the c - k -ANNS answers of q . λ is a parameter which is determined by m and n . We will discuss the settings of m and λ in Section 5.

Notably, $\text{LCCS}(H(o), H(q))$ can be considered as a dynamic concatenation of l consecutive hash values, i.e., $h_i(o)$,

$h_{i+1}(o), \dots, h_{(i+l)\%m}(o)$, where $l = |LCCS(H(o), H(q))|$. Thus, the LCCS search framework can be considered as a *dynamic concatenating search framework*. Similar to the static concatenating search framework, the false positives can be effectively avoided due to the concatenation. Furthermore, since Algorithm 2 prioritizes the objects with the largest $|LCCS(H(o), H(q))|$ as candidates, it can also identify the correct answers efficiently.

4.2 Multi-Probe LCCS-LSH

The multi-probe schemes are widely used to reduce space overhead, such as Multi-Probe LSH [31] for random projection LSH family [11] and FALCONN [3] for cross-polytope LSH family [37]. However, they are designed for the static concatenating search framework. It is inefficient to trivially adapt existing multi-probe schemes to LCCS-LSH.

Challenges. To explain the reason why existing multi-probe schemes do not work well with LCCS-LSH, we first consider a trivial multi-probe extension: given a hash string $H(q) = [h_1(q), h_2(q), \dots, h_m(q)]$, we adopt existing multi-probe schemes to (virtually) generate a sequence of probes by modifying some of $h_i(q)$ among $H(q)$; then, we conduct a λ -LCCS search of this modified $H(q)$ in the probing sequence using Algorithm 2. This trivial multi-probe extension, however, has two major problems. Firstly, if we modify a single h_i only, since Algorithm 2 uses LCP to find the objects with largest $|LCCS(H(o), H(q))|$, the LCP from most of the positions after i , i.e., $i+1, i+2, \dots$, are identical to those before modification, which should be avoided. Secondly, for the λ -LCCS search of $H(q)$ after two modifications which are far away from each other, it is very likely that the new probed objects were checked in previous probing sequence, leading to redundant computations.

Example 4.1: We now use Figure 3 for an illustration. Suppose $H(q) = [1, 2, 3, 4, 5, 6, 7, 8]$ and $H^{(1)}, H^{(2)}$, and $H^{(3)}$ are three alternative probes by modifying $h_1(q)$ and $h_4(q)$ to 5. Let $\mathcal{T} = \{H(o_1), H(o_2), H(o_3)\}$. Firstly, by modifying $H(q)$ to $H^{(1)}$, except for $shift(\mathcal{T}, 2)$ and $shift(\mathcal{T}, 3)$, the objects that have longest LCP of $H^{(1)}$ from other $shift(\mathcal{T}, i)$'s do not change, i.e., $i \in \{4, 5, 6, 7, 0, 1\}$. Thus, they should be avoided for the λ -LCCS search of $H^{(1)}$. Similarly, for $H^{(2)}$, we only need to consider $shift(\mathcal{T}, 5)$, $shift(\mathcal{T}, 6)$, $shift(\mathcal{T}, 7)$, and $shift(\mathcal{T}, 0)$. Secondly, considering $H^{(3)}$, which is a combination of $H^{(1)}$ and $H^{(2)}$ and they are far enough. We can see the new candidates introduced by $H^{(3)}$ are either the objects from $shift(\mathcal{T}, 2)$ and $shift(\mathcal{T}, 3)$ by the λ -LCCS search of $H^{(1)}$ or those from $shift(\mathcal{T}, 5)$, $shift(\mathcal{T}, 6)$, $shift(\mathcal{T}, 7)$, and $shift(\mathcal{T}, 0)$ by the λ -LCCS search of $H^{(2)}$. Since $H^{(1)}$ and $H^{(2)}$ have fewer modifications than $H^{(3)}$, they have higher priority than $H^{(3)}$. The new candidates introduced by $H^{(3)}$ were checked already. It is redundant to probe $H^{(3)}$. Δ

	h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8		h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8
o_1	1	2	4	5	6	6	7	8	$H^{(1)}$	1	2	3	5	5	6	7	8
o_2	5	2	2	4	3	6	7	8		5	2	3	4	5	6	7	8
o_3	3	1	3	5	5	6	4	9	$H^{(2)}$	5	2	3	4	5	6	7	8
q	1	2	3	4	5	6	7	8	$H^{(3)}$	5	2	3	5	5	6	7	8

Figure 3: An example of MP-LCCS-LSH

MP-LCCS-LSH. To address these two problems, we design a multi-probe scheme for LCCS-LSH, named MP-LCCS-LSH, making use of existing multi-probe schemes, such as Multi-Probe LSH and FALCONN. Given a hash string $H(q) = [h_1(q), h_2(q), \dots, h_m(q)]$, we can get m lists of alternative hash values, i.e., $\{h_1(q)^{(j)}\}, \{h_2(q)^{(j)}\}, \dots, \{h_m(q)^{(j)}\}$, where each $\{h_i(q)^{(j)}\}$ is a list of alternative hash values of $h_i(q)$. For example, for Multi-Probe LSH, $\{h_i(q)^{(j)}\} = \{h_i(q) \pm 1, h_i(q) \pm 2, \dots\}$, whereas for FALCONN, $\{h_i(q)^{(j)}\}$ is a list of other vertices of the cross-polytope. Let $score(i, h_i(q)^{(j)})$ be the score of the j^{th} alternative $h_i(q)^{(j)}$ in i^{th} position, and we reuse the score function from existing multi-probe schemes. Without loss of generality, we consider each $\{h_i(q)^{(j)}\}$ is sorted in ascending order of their scores. A perturbation vector δ is a list of pairs $(i, h_i(q)^{(j)})$, where i is the position of modification and $h_i(q)^{(j)}$ is used to replace $h_i(q)$, e.g., $\delta = \{(2, h_2(q)^{(1)}), (5, h_5(q)^{(3)})\}$ means to modify $h_2(q)$ to $h_2(q)^{(1)}$ and $h_5(q)$ to $h_5(q)^{(3)}$. We inherent $score(\delta)$ to be the score of δ from existing multi-probe schemes.

Skip Unaffected Positions. For the first problem, we skip the unaffected positions. During the first λ -LCCS search of $H(q)$, we additionally store the matched positions $pos_{l,i}, pos_{u,i}$ and the lengths $len_{l,i}, len_{u,i}$ for each i at lines 2, 7, and 9 of Algorithm 2. If the modification of $H(q)$ is not in the positions between i and $i + \max(len_{l,i}, len_{u,i})$, it will not affect the LCP of $shift(H(q), i-1)$ at position i . Thus, instead of conducting a full λ -LCCS search from position 1 to m , the probe of $H(q)$ with $\delta = \{(i_1, h_{i_1}(q)^{(j_1)}), (i_2, h_{i_2}(q)^{(j_2)}), \dots\}$ can be checked by the LCP of $shift(H(q), i)$ starting from the first position i_s such that $i_s + \max(len_{l,i_s}, len_{u,i_s}) > i_1$ to i_1 , i.e., with a modification of Algorithm 2 at lines 2 and 5.

Perturbation Vector Generation. For the second problem, we restrict the gap between two adjacent modified positions in a perturbation vector. For example, given a perturbation vector $\delta = \{(1, h_1(q)^{(3)}), (2, h_2(q)^{(1)}), (5, h_5(q)^{(2)})\}$, the gaps of δ at the 1^{st} and 2^{nd} positions are respectively $2 - 1 = 1$ and $5 - 2 = 3$, and they should be less than or equal to a threshold MAX_GAP . We set $MAX_GAP = 2$ in practice.

Based on this heuristic idea, we propose a perturbation vector generation method within the similar shift-expand operations in [31]. The pseudo-code of this method can be referred to [29]. We name them p_shift and p_expand

to distinguish from the shift operation of CSA. Let $\delta = \{(i_1, h_{i_1}(q)^{(j_1)}), (i_2, h_{i_2}(q)^{(j_2)}), \dots, (i_e, h_{i_e}(q)^{(j_e)})\}$. These two operations are defined as follows:

- $p_shift(\delta)$: use the next alternative hash value of the last modification operation of δ , i.e., $p_shift(\delta) = \{(i_1, h_{i_1}(q)^{(j_1)}), (i_2, h_{i_2}(q)^{(j_2)}), \dots, (i_e, h_{i_e}(q)^{(j_e+1)})\}$;
- $p_expand(\delta, gap)$: append $(i_e + gap, h_{i_e+gap}(q)^{(1)})$ to δ , i.e., $p_expand(\delta, gap) = \{(i_1, h_{i_1}(q)^{(j_1)}), (i_2, h_{i_2}(q)^{(j_2)}), \dots, (i_e, h_{i_e}(q)^{(j_e)}), (i_e + gap, h_{i_e+gap}(q)^{(1)})\}$.

Remarks. Even though the formulas of p_shift and p_expand are very similar to [31], the meaning of the perturbation vector is different. Following the similar proofs from [31], it can be shown that all perturbation vectors with gap less than MAX_GAP can be generated by this method, and they will be probed in ascending order of their scores.

5 THEORETICAL ANALYSIS

5.1 Quality Guarantee

We now establish a quality guarantee for LCCS-LSH. Given any two strings $T = [t_1, t_2, \dots, t_m]$ and $Q = [q_1, q_2, \dots, q_m]$, suppose the probability that $t_i = q_i$ for each i is independent and equals to p , i.e., $\Pr[t_i = q_i] = p$. Let $F_{m,p}(x) = \Pr[|LCCS(T, Q)| \leq x]$ be the CDF of the length of LCCS between T and Q . Notably, $F_{m,p}(x)$ decreases monotonically as p increases when m and x are fixed.

Let $B(q, R)$ and $\bar{B}(q, R)$ be the set of $\{o \in \mathcal{D} \mid Dist(o, q) \leq R\}$ and $\{o \in \mathcal{D} \mid Dist(o, q) > R\}$, respectively. Since $F_{m,p}(x)$ is monotonic w.r.t. p , we have Lemma 5.1 as follows.

Lemma 5.1: *Given a parameter x such that $0 < x \leq m$, for any $o_i \in \bar{B}(q, cR)$, $\Pr[|LCCS(H(o_i), H(q))| \leq x] \geq F_{m,p_2}(x)$, and for any $o_j^* \in B(q, R)$, $\Pr[|LCCS(H(o_j^*), H(q))| > x] \geq 1 - F_{m,p_1}(x)$.*

According to Lemma 5.1, if we want to demonstrate that LCCS-LSH enjoys the (R, c) -NNS with constant probability, we first need to study the property of $F_{m,p}(x)$.

According to [17], the longest consecutive heads in n coin tosses with $\Pr[Head] = p$ can be asymptotically estimated by the largest value of $n(1-p)$ i.i.d. random variables that follow the exponential distribution. Thus, for a sufficiently large m , if we follow the similar constructions to LCCS-LSH except for the first random variable, $F_{m,p}(x)$ can also be modeled by the largest value of $m(1-p)$ i.i.d. random variables that follow the exponential distribution. Hence,

Lemma 5.2: *Let $\hat{F}_p(x) = \exp(-p^x)$ be the CDF of the extreme value distribution of x . As $m \rightarrow \infty$, $\forall x$, $F_{m,p}(x) - \hat{F}_p(x - \log_{1/p}(m(1-p))) \rightarrow 0$.*

PROOF. Lemma 5.2 holds due to the Theorem 1 of [17]. It corresponds to the case when $k = 0$ [17]. $\square$

Theorem 5.1: *Given a distance metric $Dist(\cdot, \cdot)$ that admits an (R, cR, p_1, p_2) -sensitive LSH family $\mathcal{H}$, the LCCS-LSH scheme with hash length m can answer the (R, c) -NNS over $Dist(\cdot, \cdot)$ by conducting λ -LCCS search with a probability at least $1/4$, where $\lambda = m^{1-1/\rho} n(1-p_1)^{-1/\rho} (1-p_2)(\ln 2)^{1/\rho} / p_2 = O(m^{1-1/\rho} n)$ and $\rho = \ln(1/p_1) / \ln(1/p_2)$.*

The proof of Theorem 5.1 can be referred to [29].

5.2 Space and Time Complexities

LCCS-LSH is LSH-family-independent and it can handle various kinds of distance metrics. Thus, we first discuss the complexities of distance computation and the computation of hash values. For simplicity, we assume the computation of $Dist(\cdot, \cdot)$ takes $O(d)$ time. The complexity of the computation of hash values for different LSH families is different. We assume computing each hash value takes $O(\eta(d))$ time. For example, the random projection LSH family [11] takes $O(d)$ time, the cross polytope LSH family [3, 37] requires $O(d \log d)$ time, whereas the random bits sampling LSH family [23] for Hamming distance only needs $\eta(d) = O(1)$.

According to Theorem 3.1, the query time complexity of Algorithm 2 is $O(\log n + (m + \lambda) \log m)$, and the space and indexing time complexities of Algorithm 1 are $O(mn)$ and $O(mn \log(n))$, respectively. In addition, in the indexing phase of LCCS-LSH, computing nm hash values for n data objects takes $O(nm \cdot \eta(d))$ time. In the query phase of LCCS-LSH, computing m hash values for each query takes $O(m \cdot \eta(d))$ time and computing the actual distance for λ candidates takes $O(\lambda d)$ time. According to Theorem 5.1, λ is determined by m and n . By setting different m values, LCCS-LSH has different time and space complexities. Thus, we introduce a parameter α to control the value of m in different scales. According to Theorems 3.1 and 5.1, we have

Corollary 5.1: *For any $0 \leq \alpha \leq \frac{1}{1-\rho}$, setting $m = O(n^{\alpha\rho})$, LCCS-LSH can answer the (R, c) -NNS with a probability at least $1/4$ using $O(n^{1+\alpha\rho})$ space, $O(n^{1+\alpha\rho}(\eta(d) + \log n))$ indexing time, and $O(n^{\alpha\rho}(\eta(d) + \alpha \log n) + n^{\alpha(\rho-1)+1}(d + \alpha \log n))$ query time.*

The upper-bound of α is $\frac{1}{1-\rho}$, because λ is at least 1. There are three typical settings of α : (i) $\alpha = 0$: the query time complexity of LCCS-LSH is equivalent to the complexity of linear scan; (ii) $\alpha = 1$: compared with E2LSH [11] and C2LSH [15], LCCS-LSH enjoys the least query time complexity; moreover, LCCS-LSH has the same space complexity as E2LSH and its index time complexity is also lower than that of E2LSH; C2LSH enjoys the least space and indexing time complexities, but its query time complexity is the largest among the three methods; (iii) $\alpha = \frac{1}{1-\rho}$: LCCS-LSH verifies only constant number of candidates, and thus it is suitable to the case that computing hash values is much cheaper than computing actual distances, e.g., the random bits sampling

Table 1: Space and time complexities of E2LSH, C2LSH, and LCCS-LSH under different settings of α

Methods	α	m	λ	Space Complexity	Indexing Time Complexity	Query Time Complexity
E2LSH [11]	–	–	–	$O(n^{1+\rho})$	$O(n^{1+\rho}\eta(d) \log n)$	$O(n^\rho(\eta(d) \log n + d))$
C2LSH [15]	–	–	–	$O(n \log n)$	$O(n \log n(\eta(d) + \log n))$	$O(n \log n)$
LCCS-LSH	0	$O(1)$	$O(n)$	$O(n)$	$O(n(\eta(d) + \log n))$	$O(nd)$
	1	$O(n^\rho)$	$O(n^\rho)$	$O(n^{1+\rho})$	$O(n^{1+\rho}(\eta(d) + \log n))$	$O(n^\rho(\eta(d) + d + \log n))$
	$\frac{1}{1-\rho}$	$O(n^{\frac{\rho}{1-\rho}})$	$O(1)$	$O(n^{\frac{1}{1-\rho}})$	$O(n^{\frac{1}{1-\rho}}(\eta(d) + \log n))$	$O(n^{\frac{\rho}{1-\rho}}(\eta(d) + \log n) + d)$

LSH family for Hamming distance in a very high dimensional space. Setting α in between 0 and $\frac{1}{1-\rho}$ can smoothly control the trade-off between space and time complexities. We summarize the space and time complexities of E2LSH, C2LSH, and LCCS-LSH in Table 1.

6 EXPERIMENTS

We study the performance of LCCS-LSH and MP-LCCS-LSH over five real-life datasets for high-dimensional c -ANNS. All methods are implemented in C++ and are compiled with g++ 8.3 using -O3 optimization. We conduct all experiments in a single thread on a machine with 8 Intel i7-3820 @ 3.60GHz CPUs and 64 GB RAM, running on Ubuntu 16.04.

6.1 Experimental Setup

Datasets and Queries. We use five real-life datasets Msong,¹ Sift,² Gist,³ GloVe,⁴ and Deep⁵ in our experiments to cover a wide range of data types, including audio, image, text, and deep-learning data. We randomly select 100 objects from their test sets and use them as queries. We refer readers to [29] for more details of the datasets and queries.

Evaluation Metrics. We use recall, overall ratio (or simply ratio), query time, index size and indexing time for performance evaluation. For the c - k -ANNS, the ratio is defined as $\frac{1}{k} \sum_{i=1}^k \frac{\text{Dist}(o_i, q)}{\text{Dist}(o_i^*, q)}$, where o_i is the i^{th} nearest object returned by a method and o_i^* is the exact i^{th} NN, $i \in \{1, 2, \dots, k\}$. We report the average recall and ratio over all queries, and we run each method for each experiment five times to report its average running time and indexing overhead.

Benchmark Methods. Since LCCS-LSH is independent of LSH family and it supports various kinds of distance metrics, we conduct experiments under two popular distance metrics: Euclidean and Angular distances. We select several state-of-the-art LSH schemes as benchmarks, namely Multi-Probe LSH [12, 31], FALCONN[3], E2LSH [1], C2LSH [15], SRS [35], and QALSH [21]. E2LSH and C2LSH are designed for Euclidean distance. We also adapt them for Angular distance.

For the c - k -ANNS, we consider $k = 10$. To make a fair comparison, we fix the maximum number of LSH functions for all methods. Specifically, we set $K \in \{1, 2, 3, \dots, 10\}$ and $L \in \{8, 16, 32, \dots, 512\}$ for E2LSH, Multi-Probe LSH, and FALCONN such that $KL \leq 512$; we set $m \in \{8, 16, 32, \dots, 512\}$ and $l \in \{2, 3, \dots, 10\}$ for C2LSH; we set the projected dimensions $d' \in \{4, 5, \dots, 10\}$ for SRS; for QALSH, we adopt QALSH⁺⁶ and set $c \in \{2.0, 3.0, 4.0\}$ for each block to build QALSH. For LCCS-LSH and MP-LCCS-LSH, we set $m \in \{8, 16, 32, \dots, 512\}$ and $\#probes \in \{1, m+1, 2m+1, 4m+1, 8m+1\}$. w is fine-tuned for the random projection LSH family,⁷ so that all methods achieve their best performance.

6.2 Results and Analysis

We study the performance of LCCS-LSH and MP-LCCS-LSH in two aspects: query performance and indexing performance. We also study their performance on the sensitivity to k and the impacts of m and $\#probes$. We refer readers to [29] for more details of those results.

Query Performance. We first study the query performance of LCCS-LSH and MP-LCCS-LSH. Different number of candidates and probes are used for all methods to achieve different recall levels. To remove the impact of parameters for each method, we report their lowest query time for all combinations of parameters under each certain recall level using grid search. The results are shown in Figures 4 and 5.

From Figure 4, we observe LCCS-LSH and MP-LCCS-LSH achieve the best or nearly the best performance under Euclidean distance. Compared with E2LSH, QALSH, and Multi-Probe LSH, even though they are close to each other over Gist and GloVe, LCCS-LSH and MP-LCCS-LSH achieve around 240% acceleration over Msong, 70% acceleration over Sift, and 80% acceleration over Deep under certain recall level. These results also verify the efficiency of CSA for the LCCS search framework in the sense that identifying objects with longest LCCS is as efficient as hash table lookups. Furthermore, Multi-Probe LSH enjoys a slightly better trade-off between efficiency and accuracy than E2LSH, which satisfies

¹<http://www.ifs.tuwien.ac.at/mir/msd/download.html>

²<http://corpus-texmex.irisa.fr/>

³<http://corpus-texmex.irisa.fr/>

⁴<https://nlp.stanford.edu/projects/glove/>

⁵https://github.com/DBWangGroupUNSW/nns_benchmark

⁶We use kd-tree to split dataset into blocks and set $leaf = 20,000$. We set $L \in \{10, 20, 30, 40\}$ projections and $L * M \in \{100, 240, 400\}$ boundary objects for each block as representative objects to determine close blocks.

⁷Specifically, w is set to be 18.75, 226.0, 11294.0, 4.65, and 0.66 for the datasets Msong, Sift, Gist, GloVe, and Deep, respectively.

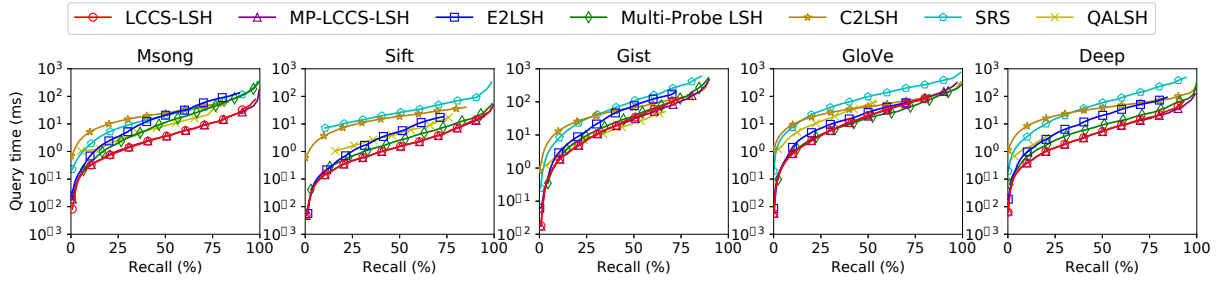


Figure 4: Query time-recall curves under Euclidean distance (lower is better)

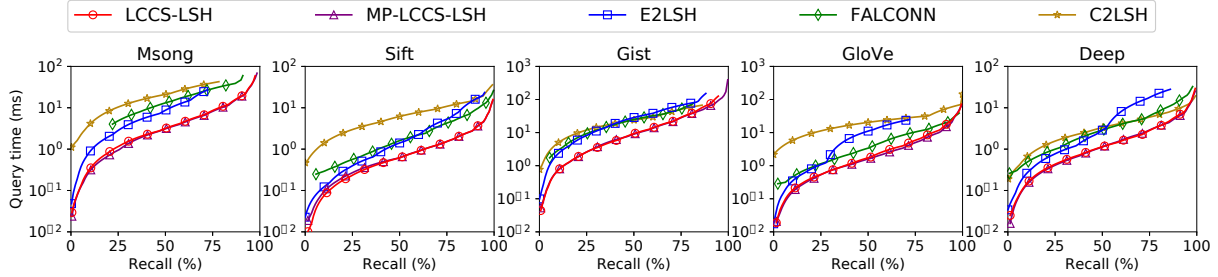


Figure 5: Query time-recall curves under Angular distance (lower is better)

the observations from [1, 11]. Compared with C2LSH and SRS, both LCCS-LSH and MP-LCCS-LSH achieve at least one order of magnitude acceleration under certain recall level for all of the five datasets, because the query time complexity of C2LSH is much worse than that of LCCS-LSH and SRS makes use of tree-based method to retrieve the candidates which is not as efficient as CSA. The performance of LCCS-LSH and MP-LCCS-LSH are close to each other. This is because although MP-LCCS-LSH introduces more probing overhead, it checks for fewer candidates than LCCS-LSH at the same recall level due to its intelligent probing strategy.

Figure 5 shows that the performance of LCCS-LSH and MP-LCCS-LSH under Angular distance is better than those of other methods among all datasets, and their advantages are more apparent. Specifically, LCCS-LSH and MP-LCCS-LSH achieve at least 100% acceleration compared with the second fastest competitor under 50% recall level for all datasets. Furthermore, the performance of FALCONN is slightly better than that of E2LSH, especially when the recall level is high, which also fits the observations from [3]. The query time-ratio curves show similar trends to the query time-recall curves. To be concise, we omit those results here.

Indexing Performance. We then study the indexing performance of LCCS-LSH and MP-LCCS-LSH. Since different parameters are used at different recall levels, we present the lowest query time under different index size (or indexing time) of all methods at 50% recall level, to show the trade-off between query time and index size (or indexing time). The results under Euclidean distance are displayed in Figure 6. Similar trends can be observed from Angular distance.

Figure 6 shows that MP-LCCS-LSH enjoys better trade-off between query time and indexing overhead than LCCS-LSH, especially when only few memory is used. This means that the intelligent probing strategy of MP-LCCS-LSH can help to produce more candidates efficiently when m is relatively small. Among all of the seven methods, Multi-Probe LSH is competitive in terms of the trade-off between query time and index size as it is designed to save space without losing too much information. For Gist and GloVe, Multi-Probe LSH uses less indexing overhead than MP-LCCS-LSH under the same query time, whereas for Msong, Sift, and Deep, MP-LCCS-LSH takes less query time under the same indexing budget if more memory is allowed. Compared to other competitors, MP-LCCS-LSH enjoys a better trade-off between query time and indexing overhead. The reasons are as follows: due to the static concatenating search framework, E2LSH cannot share LSH functions between different hash tables, which leads to a large indexing overhead; C2LSH, SRS, and QALSH can achieve good performance with small indexing budget, but they cannot significantly reduce the query time when more memory is allowed.

We also observe that at 50% recall level, increasing index size (and indexing time) cannot always reduce the query time of all methods, because certain index size is good enough to achieve 50% recall. In this case, using more hash functions will introduce more memory and more indexing time.

7 RELATED WORK

LSH is one of the most popular methods for high-dimensional NNS. It was originally introduced by Indyk and Motwani [23] for Hamming space, and later was extended to other

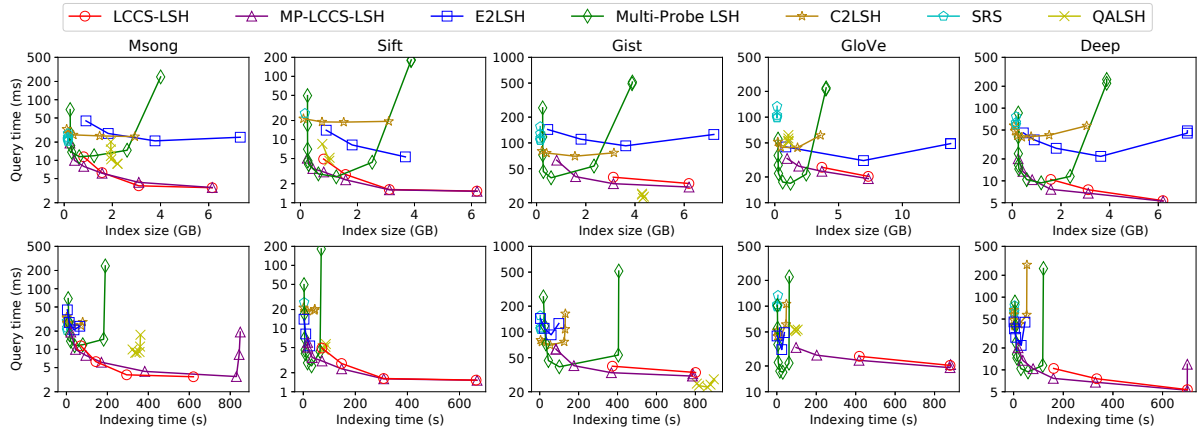


Figure 6: Query time-Index size/Indexing time curves at 50% recall level under Euclidean distance.

distance metrics such as Jaccard similarity [8], Angular distance [10, 37], and l_p distance [11] later. Although extensive studies have been done for the LSH families, the search framework behind the LSH families is less investigated. Existing works for the search framework can be roughly divided into two categories: static concatenating search framework and dynamic collision counting framework and their variants.

The first category is the static concatenating search framework and its variants. This search framework is most widely used in the LSH literatures. There are two potential problems behind this search framework. Firstly, the setting of K is sensitive to R , and hence it needs to be tuned for every dataset. Secondly, the theoretical L is usually prohibitively large. Thus, many variants of this search framework have been proposed to address these two issues.

To make K suitable for different R values and datasets, LSH-Forest [5] concatenates hash values into a sequence instead of a single hash value, so that the LCP between the hash values of query and data objects can be found via a trie structure. LSB-Forest [36] uses a z-order curve to encode the hash values and uses a B-tree to index the hash codes. Hence, the K value can be conceptually automatically decided for different R . Similarly, SK-LSH [30] sorts the compound keys in alphabetical order, and thus it can reduce the I/O costs for external storages. Compared with these methods, LCCS-LSH uses the data structure CSA to store hash values. Since CSA can reuse the hash values in every position, it carries more information than sequence and curves. From this perspective, LCCS-LSH can be considered to extend them by virtually building more trees. To reduce the large L , Multi-Probe LSH [31] is proposed to heuristically boost conceptual L by probing more buckets without extra memory usage. FALCONN [3] further demonstrates its effectiveness on another LSH family. We also propose a new Multi-Probe scheme MP-LCCS-LSH to boost the conceptual L and reduce the indexing overhead.

Another category is the dynamic collision counting framework. C2LSH [15] uses the number of identical hash values that data objects and query collide as the indicator of their actual distance. QALSH [21, 22] further extends this idea by considering real number as “hash value.” The counting-based indicator, although can identify the near neighbors of query precisely, unavoidably requires to count a large number of false positives, which limits its scalability when n is very large. In contrast, LCCS-LSH can be understood as a dynamic concatenating search framework. By leveraging CSA for k -LCCS search, LCCS-LSH is able to answer c -ANNS with *sublinear* query time and *sub-quadratic* space.

8 CONCLUSION

In this paper, we introduce a novel LSH scheme LCCS-LSH for high-dimensional c -ANNS with a theoretical guarantee. We define a new concept of LCCS and propose a novel data structure CSA for k -LCCS search. CSA is potentially of separate interest for other fields of computer science. LCCS-LSH adopts the LCCS search framework to dynamically concatenate consecutive hash values, which yields a simple yet effective way to identify the close objects. It requires to tune a single parameter m only, which is unavoidable for the trade-off between space and query time. In addition, we propose MP-LCCS-LSH to further reduce the indexing overhead. Extensive experiments over five real-life datasets demonstrate the superior performance of LCCS-LSH and MP-LCCS-LSH.

ACKNOWLEDGMENTS

This research is supported by the National Research Foundation, Singapore under its Strategic Capability Research Centres Funding Initiative and the National Research Foundation Singapore under its AI Singapore Programme. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

REFERENCES

- [1] Alexandr Andoni. 2005. E2LSH 0.1 User manual. <http://web.mit.edu/andoni/www/LSH/index.html> (2005).
- [2] Alexandr Andoni and Piotr Indyk. 2006. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. In *FOCS*. 459–468.
- [3] Alexandr Andoni, Piotr Indyk, Thijs Laarhoven, Ilya Razenshteyn, and Ludwig Schmidt. 2015. Practical and optimal LSH for angular distance. In *NeurIPS*. 1225–1233.
- [4] Alexandr Andoni and Ilya Razenshteyn. 2015. Optimal data-dependent hashing for approximate near neighbors. In *STOC*. 793–801.
- [5] Mayank Bawa, Tyson Condie, and Prasanna Ganesan. 2005. LSH forest: self-tuning indexes for similarity search. In *WWW*. 651–660.
- [6] Jon Louis Bentley. 1990. K-d trees for semidynamic point sets. In *SoCG*. 187–197.
- [7] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *ICML*. 97–104.
- [8] Andrei Z Broder. 1997. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of Sequences*. 21–29.
- [9] Andrei Z Broder, Moses Charikar, Alan M Frieze, and Michael Mitzenmacher. 1998. Min-wise independent permutations. In *STOC*. 327–336.
- [10] Moses S Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *STOC*. 380–388.
- [11] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *SoCG*. 253–262.
- [12] Wei Dong, Zhe Wang, William Josephson, Moses Charikar, and Kai Li. 2008. Modeling LSH for performance tuning. In *CIKM*. 669–678.
- [13] Ronald Fagin, Ravi Kumar, and Dandapani Sivakumar. 2003. Efficient similarity search and classification via rank aggregation. In *SIGMOD*. 301–312.
- [14] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *PVLDB* 12, 5 (2019), 461–474.
- [15] Junhao Gan, Jianlin Feng, Qiong Fang, and Wilfred Ng. 2012. Locality-sensitive hashing scheme based on dynamic collision counting. In *SIGMOD*. 541–552.
- [16] Aristides Gionis, Piotr Indyk, Rajeev Motwani, et al. 1999. Similarity search in high dimensions via hashing. In *Vldb*, Vol. 99. 518–529.
- [17] Louis Gordon, Mark F Schilling, and Michael S Waterman. 1986. An extreme value theory for long head runs. *Probability Theory and Related Fields* 72, 2 (1986), 279–287.
- [18] Antonin Guttman. 1984. R-trees: A dynamic index structure for spatial searching. In *SIGMOD*. 47–57.
- [19] Sarel Har-Peled, Piotr Indyk, and Rajeev Motwani. 2012. Approximate nearest neighbor: Towards removing the curse of dimensionality. *Theory of Computing* 8, 1 (2012), 321–350.
- [20] Alexander Hinneburg, Charu C Aggarwal, and Daniel A Keim. 2000. What is the nearest neighbor in high dimensional spaces?. In *Vldb*. 506–515.
- [21] Qiang Huang, Jianlin Feng, Qiong Fang, Wilfred Ng, and Wei Wang. 2017. Query-aware locality-sensitive hashing scheme for l_p norm. *VldbJ* 26, 5 (2017), 683–708.
- [22] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *PVLDB* 9, 1 (2015), 1–12.
- [23] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *STOC*. 604–613.
- [24] Hosagrahar V Jagadish, Beng Chin Ooi, Kian-Lee Tan, Cui Yu, and Rui Zhang. 2005. iDistance: An adaptive B+-tree based indexing method for nearest neighbor search. *TODS* 30, 2 (2005), 364–397.
- [25] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *TPAMI* 33, 1 (2010), 117–128.
- [26] Norio Katayama and Shin'ichi Satoh. 1997. The SR-tree: An index structure for high-dimensional nearest neighbor queries. *ACM SIGMOD Record* 26, 2 (1997), 369–380.
- [27] Jon M Kleinberg. 1997. Two algorithms for nearest-neighbor search in high dimensions. In *STOC*, Vol. 97. 599–608.
- [28] Yifan Lei, Qiang Huang, Mohan Kankanhalli, and Anthony Tung. 2019. Sublinear Time Nearest Neighbor Search over Generalized Weighted Space. In *ICML*. 3773–3781.
- [29] Yifan Lei, Qiang Huang, Mohan Kankanhalli, and Anthony Tung. 2020. Locality-Sensitive Hashing Scheme based on Longest Circular Co-Substring. *arXiv preprint arXiv:2004.05345* (2020).
- [30] Yingfan Liu, Jiangtao Cui, Zi Huang, Hui Li, and Heng Tao Shen. 2014. SK-LSH: an efficient index structure for approximate nearest neighbor search. *PVLDB* 7, 9 (2014), 745–756.
- [31] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-probe LSH: efficient indexing for high-dimensional similarity search. In *Vldb*. 950–961.
- [32] Yury A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *TPAMI* (2018).
- [33] Udi Manber and Gene Myers. 1993. Suffix arrays: a new method for on-line string searches. *SICOMP* 22, 5 (1993), 935–948.
- [34] Rina Panigrahy. 2006. Entropy based nearest neighbor search in high dimensions. In *SODA*. 1186–1195.
- [35] Yifang Sun, Wei Wang, Jianbin Qin, Ying Zhang, and Xuemin Lin. 2014. SRS: solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index. *PVLDB* 8, 1 (2014), 1–12.
- [36] Yufei Tao, Ke Yi, Cheng Sheng, and Panos Kalnis. 2009. Quality and efficiency in high dimensional nearest neighbor search. In *SIGMOD*. 563–576.
- [37] Kengo Terasawa and Yuzuru Tanaka. 2007. Spherical lsh for approximate nearest neighbor search on unit hypersphere. In *Workshop on Algorithms and Data Structures*. 27–38.
- [38] Yiqiu Wang, Anshumali Shrivastava, Jonathan Wang, and Junghee Ryu. 2018. Randomized Algorithms Accelerated over CPU-GPU for Ultra-High Dimensional Similarity Search. In *SIGMOD*. 889–903.
- [39] Roger Weber, Hans-Jörg Schek, and Stephen Blott. 1998. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *Vldb*, Vol. 98. 194–205.
- [40] Yuxin Zheng, Qi Guo, Anthony KH Tung, and Sai Wu. 2016. LazyLSH: Approximate nearest neighbor search for multiple distance functions with a single index. In *SIGMOD*. 2023–2037.
- [41] Jingbo Zhou, Qi Guo, HV Jagadish, Lubos Krcaľ, Siyuan Liu, Wenhao Luan, Anthony KH Tung, Yueji Yang, and Yuxin Zheng. 2018. A generic inverted index framework for similarity search on the GPU. In *ICDE*. 893–904.